

大作业: 上机报告

王涵予 2300010705

2026 年 1 月 13 日

目录

1 问题描述: Stokes方程与交错网格上的MAC格式	2
1.1 Stokes方程	2
1.2 MAC格式离散	2
1.3 数值算例	4
2 P1: 使用 DGS 磨光子	5
2.1 数值方法	5
2.1.1 V-cycle多重网格法	5
2.1.2 DGS迭代法	6
2.1.3 限制算子和提升算子	9
2.2 具体算法实现	9
2.3 数值结果与分析	11
3 P2: 使用 Uzawa Iteraion Method	14
3.1 数值方法	14
3.1.1 实用共轭梯度法	14
3.1.2 Uzawa Iteraion Method	14
3.2 具体算法实现	15
3.3 数值结果	16
4 P3: 使用 Inexact Uzawa Iteration Method	18
4.1 数值方法	18
4.1.1 预优共轭梯度法	18
4.1.2 Inexact Uzawa Iteraion Method	19
4.2 具体算法实现	19
4.3 数值结果及相应分析	21

1 问题描述: Stokes方程与交错网格上的MAC格式

1.1 Stokes方程

考虑Stokes方程

$$\begin{cases} -\Delta \vec{u} + \nabla p = \vec{F}, & (x, y) \in (0, 1) \times (0, 1) \\ \text{div } \vec{u} = 0, & (x, y) \in (0, 1) \times (0, 1) \end{cases}$$

边界条件为

$$\begin{aligned} \frac{\partial u}{\partial \vec{n}} &= b, & y=0, & \quad \frac{\partial u}{\partial \vec{n}} = b, & y=1 \\ \frac{\partial v}{\partial \vec{n}} &= l, & x=0, & \quad \frac{\partial v}{\partial \vec{n}} = r, & x=1 \\ u &= 0, & x=0, 1, & \quad v = 0, & y=0, 1 \end{aligned}$$

其中, $\vec{u} = (u, v)$ 为速度, p 为压力, $\vec{F} = (f, g)$ 为外力, $\vec{n}$ 为外法向向量

1.2 MAC格式离散

利用交错网格上的MAC格式离散Stokes方程, 记 $h = \frac{1}{N}$

$$U_{(N+1) \times N} = (U_{ij}), \quad U_{ij} \longrightarrow u(x_i, y_j), \quad x_i = (i-1)h, y_i = (j - \frac{1}{2})h$$

$$V_{N \times (N+1)} = (V_{ij}), \quad V_{ij} \longrightarrow v(x_i, y_j), \quad x_i = (i - \frac{1}{2})h, y_i = (j-1)h$$

$$P_{N \times N} = (P_{ij}), \quad P_{ij} \longrightarrow p(x_i, y_j), \quad x_i = (i - \frac{1}{2})h, y_i = (j - \frac{1}{2})h$$

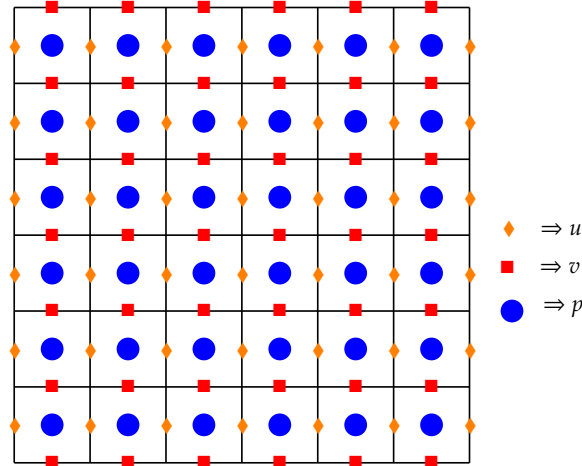


图 1: MAC离散(此处图片来自讲义)

可得如下方程

$$if \quad 2 \leq i \leq N, 2 \leq j \leq N-1, \quad x_i = (i-1)h, y_i = (j - \frac{1}{2})h$$

$$-\frac{U_{i-1,j} - 2U_{i,j} + U_{i+1,j}}{h^2} - \frac{U_{i,j-1} - 2U_{i,j} + U_{i,j+1}}{h^2} + \frac{P_{i,j} - P_{i-1,j}}{h} = f(x_i, y_j)$$

$$\text{if } 2 \leq i \leq N, \quad x_i = (i-1)h$$

$$\begin{aligned} & -\frac{U_{i-1,1} - 2U_{i,1} + U_{i+1,1}}{h^2} - \frac{U_{i,2} - U_{i,1}}{h^2} - \frac{b(x_i)}{h} + \frac{P_{i,1} - P_{i-1,1}}{h} = f(x_i, \frac{1}{2}h) \\ & -\frac{U_{i-1,N} - 2U_{i,N} + U_{i+1,N}}{h^2} - \frac{U_{i,N} - U_{i,N-1}}{h^2} - \frac{t(x_i)}{h} + \frac{P_{i,1} - P_{i-1,1}}{h} = f(x_i, (N - \frac{1}{2})h) \end{aligned}$$

$$\text{if } i = 1 \text{ or } N + 1$$

$$U_{i,j} = 0 \quad \forall j$$

$$\text{if } 2 \leq j \leq N, 2 \leq i \leq N-1, \quad x_i = (i - \frac{1}{2})h, y_i = (j-1)h$$

$$-\frac{V_{i-1,j} - 2V_{i,j} + V_{i+1,j}}{h^2} - \frac{V_{i,j-1} - 2V_{i,j} + V_{i,j+1}}{h^2} + \frac{P_{i,j} - P_{i,j-1}}{h} = g(x_i, y_j)$$

$$\text{if } 2 \leq j \leq N, \quad y_j = (j-1)h$$

$$\begin{aligned} & -\frac{V_{1,i-1} - 2V_{1,i} + V_{1,i+1}}{h^2} - \frac{V_{2,i} - V_{1,i}}{h^2} - \frac{l(x_i)}{h} + \frac{P_{1,i} - P_{1,i-1}}{h} = g(\frac{1}{2}h, y_j) \\ & -\frac{V_{N,i-1} - 2V_{N,i} + V_{N,i+1}}{h^2} - \frac{V_{N,i} - V_{N-1,i}}{h^2} - \frac{r(x_i)}{h} + \frac{P_{1,i} - P_{1,i-1}}{h} = g((N - \frac{1}{2})h, y_j) \end{aligned}$$

$$\text{if } j = 1 \text{ or } N + 1$$

$$V_{i,j} = 0 \quad \forall j$$

$$\text{if } 1 \leq i \leq N, 1 \leq j \leq N$$

$$\frac{U_{i+1,j} - U_{i,j}}{h} + \frac{V_{i,j+1} - V_{i,j}}{h} = 0$$

将 b, t, l, r 移动至 RHS , 并对系数做处理使得满足对称关系, 做离散

$$F_{(N+1) \times N} = (F_{ij}), \quad F_{ij} \longrightarrow f(x_i, y_j) (\text{边界处考虑 } b, t), \quad x_i = (i-1)h, y_i = (j - \frac{1}{2})h$$

$$G_{N \times (N+1)} = (G_{ij}), \quad G_{ij} \longrightarrow g(x_i, y_j) (\text{边界处考虑 } l, r), \quad x_i = (i - \frac{1}{2})h, y_i = (j-1)h$$

最终得到线性方程如下

$$\text{if } 2 \leq i \leq N, 2 \leq j \leq N-1$$

$$-\frac{U_{i-1,j} - 2U_{i,j} + U_{i+1,j}}{h^2} - \frac{U_{i,j-1} - 2U_{i,j} + U_{i,j+1}}{h^2} + \frac{P_{i,j} - P_{i-1,j}}{h} = F_{i,j}$$

$$\text{if } 2 \leq i \leq N$$

$$-\frac{U_{i-1,1} - 2U_{i,1} + U_{i+1,1}}{h^2} - \frac{U_{i,2} - U_{i,1}}{h^2} + \frac{P_{i,1} - P_{i-1,1}}{h} = F_{i,1}$$

$$-\frac{U_{i-1,N} - 2U_{i,N} + U_{i+1,N}}{h^2} - \frac{U_{i,N} - U_{i,N-1}}{h^2} + \frac{P_{i,1} - P_{i-1,1}}{h} = F_{i,N}$$

$$\text{if } i = 1 \text{ or } N + 1$$

$$U_{i,j} = 0 \quad \forall j$$

$$\text{if } 2 \leq j \leq N, 2 \leq i \leq N-1$$

$$-\frac{V_{i-1,j} - 2V_{i,j} + V_{i+1,j}}{h^2} - \frac{V_{i,j-1} - 2V_{i,j} + V_{i,j+1}}{h^2} + \frac{P_{i,j} - P_{i,j-1}}{h} = G_{i,j}$$

$$\text{if } 2 \leq j \leq N$$

$$-\frac{V_{1,i-1} - 2V_{1,i} + V_{1,i+1}}{h^2} - \frac{V_{2,i} - V_{1,i}}{h^2} + \frac{P_{1,i} - P_{1,i-1}}{h} = G_{1,j}$$

$$-\frac{V_{N,i-1} - 2V_{N,i} + V_{N,i+1}}{h^2} - \frac{V_{N,i} - V_{N-1,i}}{h^2} + \frac{P_{1,i} - P_{1,i-1}}{h} = G_{N,j}$$

if $j = 1 \text{ or } N + 1$

$$V_{i,j} = 0 \quad \forall j$$

if $1 \leq i \leq N, 1 \leq j \leq N$

$$-\frac{U_{i+1,j} - U_{i,j}}{h} - \frac{V_{i,j+1} - V_{i,j}}{h} = 0$$

其可以写成格式

$$\begin{pmatrix} A & B \\ B^T & 0 \end{pmatrix} \begin{pmatrix} X \\ P \end{pmatrix} = \begin{pmatrix} Y \\ 0 \end{pmatrix}$$

其中, X 是 U, V 的向量化, Y 是 F, G 的向量化, 通过改变最后一式的符号, 保证了对称性

1.3 数值算例

在 $\Omega = (0, 1) \times (0, 1)$ 上

$$f(x, y) = -4\pi^2(2\cos(2\pi x) - 1)\sin(2\pi y) + x^2$$

$$g(x, y) = 4\pi^2(2\cos(2\pi y) - 1)\sin(2\pi x)$$

$$b(x) = -2\pi(1 - \cos(2\pi x))$$

$$t(x) = 2\pi(1 - \cos(2\pi x))$$

$$l(y) = 2\pi(1 - \cos(2\pi y))$$

$$r(y) = -2\pi(1 - \cos(2\pi y))$$

真解为

$$u(x, y) = (1 - \cos(2\pi x))\sin(2\pi y)$$

$$v(x, y) = -(1 - \cos(2\pi y))\sin(2\pi x)$$

$$p(x, y) = \frac{x^3}{3} - \frac{1}{12}$$

2 P1: 使用 DGS 磨光子

2.1 数值方法

2.1.1 V-cycle多重网格法

用V-cycle求解方程

$$\begin{aligned} AV &= F, \quad A \in \mathbb{R}^{N \times N}, V, F \in \mathbb{R}^{N \times 1} \\ A_h &= A, V_h = V, F_h = F, \quad h = \frac{1}{N}, \quad N = 2^n \end{aligned}$$

step1: 用DGS迭代法, 以 $V_h^{(0)} = 0$ 为初值, 做 v_1 次迭代得 $V_h^{(v_1)}$

$$R_h = F_h - A_h V_h^{v_1}$$

step2: 令

$$A_{2h} = I_h^{2h} A_h I_{2h}^h$$

其中 I_h^{2h} 为从 N 到 $\frac{1}{2}N$ 的限制算子, I_{2h}^h 为从 $\frac{1}{2}N$ 到 N 的提升算子

特别地, 对于原问题, 我们取 A_{2h} 为 $\frac{1}{2}N$ 时的 MAC 离散化所得(此处参考提示5)

令

$$R_{2h} = I_h^{2h} R_h$$

解方程

$$A_{2h} V_{2h} = R_{2h}$$

以 $V_{2h}^{(0)}$ 为初值(第1个 V-cycle 中取为0), 做 v_1 次 DGS 迭代, 得 $V_{2h}^{(v_1)}$ 以及

$$R_{2h} = F_{2h} - A_{2h} V_{2h}^{(v_1)}$$

step(n-1): 得到 $V_{2^{n-l-1}h}^{(v_1)}$, 以及

$$R_{2^{n-l-1}h} = F_{2^{n-l-1}h} - A_{2^{n-l-1}h} V_{2^{n-l-1}h}^{(v_1)}$$

step(n-l+1): 令

$$A_{2^{n-l}h} = I_{2^{n-l-1}h}^{2^{n-l}h} A_{2^{n-l-1}h} I_{2^{n-l-1}h}^{2^{n-l}h}$$

并令

$$F_{2^{n-l}h} = I_{2^{n-l-1}h}^{2^{n-l}h} R_{2^{n-l-1}h}$$

直接求解

$$A_{2^{n-l}h} V_{2^{n-l}h} = F_{2^{n-l}h}$$

得 $V_{2^{n-l}h}$

step(n-l)': 计算

$$V_{2^{n-l-1}h}^{(v_1+1)} = V_{2^{n-l-1}h}^{(v_1)} + I_{2^{n-l}h}^{2^{n-l-1}h} V_{2^{n-l}h}$$

以 $V_{2^{n-l-1}h}^{(v_1+1)}$ 为初值, 求解

$$A_{2^{n-l-1}h} V_{2^{n-l-1}h} = F_{2^{n-l-1}h}$$

做 v_2 次G-S迭代, 得 $V_{2^{n-l-1}h}^{(v_1+v_2+1)}$

step1': 计算

$$V_h^{(v_1+1)} = V_h^{(v_1)} + I_{2h}^h V_{2h}$$

以 $V_h^{(v_1+1)}$ 为初值, 求解

$$A_h V_h = F_h$$

做 v_2 次 DGS 迭代, 得 $V_h^{(v_1+v_2+1)}$

计算

$$r_h = F_h - A_h V_h^{(v_1+v_2+1)}, \quad r_0 = F_h - A_h V_h^{(0)}$$

停止条件为

$$\frac{\|r_h\|_2}{\|r_0\|_2} < 10^{-8}$$

如满足条件, 停止迭代, $V_h^{(v_1+v_2+1)}$ 为近似解

否则回到 step1, 以 $V_h^{(0)} = V_h^{(v_1+v_2+1)}$ 为初值做 V-cycle

2.1.2 DGS 迭代法

用 DGS 方法对方程

$$\begin{pmatrix} A & B \\ B^T & 0 \end{pmatrix} \begin{pmatrix} X \\ P \end{pmatrix} = \begin{pmatrix} Y \\ 0 \end{pmatrix}$$

做一次迭代

step1: 先对 X 做一次 Gauss-Seidel 迭代

$$R \leftarrow Y - (AX + BP)$$

$$R \rightarrow RF, RG$$

用 RF, RG , 分别进行逐个更新, 得到 EU, EV , 其中, 对 U 按 $j = 1 : N, i = 1 : N + 1$ 顺序更新, 对 V 按 $j = 1 : N, i = 1 : N + 1$ 顺序更新, 具体地从方程导出迭代格式

$j = 1, i = 1$

$$U_{1,1} = 0 \implies EU_{1,1} = RF_{1,1}$$

$j = 1, i = 2 : N$

$$-\frac{U_{i-1,1} - 2U_{i,1} + U_{i+1,1}}{h^2} - \frac{U_{i,2} - U_{i,1}}{h^2} + \frac{P_{i,1} - P_{i-1,1}}{h} = F(i, 1)$$

$$\implies -\frac{EU_{i-1,1} - 2EU_{i,1}}{h^2} - \frac{-EU_{i,1}}{h^2} = RF(i, 1)$$

$$\implies EU_{i,1} = \frac{1}{3}EU_{i-1,1} + \frac{1}{3}h^2 RF_{i,1}$$

$j = 1, i = N + 1,$

$$U_{N+1,1} = 0 \implies EU_{N+1,1} = RF_{N+1,1}$$

$j = 2 : N - 1, i = 1$

$$U_{1,j} = 0 \implies EU_{1,j} = RF_{1,j}$$

$j = 2 : N - 1, i = 2 : N$

$$-\frac{U_{i-1,j} - 2U_{i,j} + U_{i+1,j}}{h^2} - \frac{U_{i,j-1} - 2U_{i,j} + U_{i,j+1}}{h^2} + \frac{P_{i,j} - P_{i-1,j}}{h} = F_{i,j}$$

$$\implies -\frac{EU_{i-1,j} - 2EU_{i,j}}{h^2} - \frac{EU_{i,j-1} - 2EU_{i,j}}{h^2} = RF_{i,j}$$

$$\Rightarrow EU_{i,j} = \frac{1}{4}EU_{i,j-1} + \frac{1}{4}EU_{i-1,j} + \frac{1}{4}h^2RF_{i,j}$$

$$j = 2 : N - 1, i = N + 1$$

$$U_{N+1,j} = 0 \Rightarrow EU_{N+1,j} = RF_{N+1,j}$$

$$j = N, i = 1$$

$$U_{i,N} = 0 \Rightarrow EU_{i,N} = RF_{i,N}$$

$$j = N, i = 2 : N$$

$$-\frac{U_{i-1,N} - 2U_{i,N} + U_{i+1,N}}{h^2} - \frac{U_{i,N} - U_{i,N-1}}{h^2} + \frac{P_{i,1} - P_{i-1,1}}{h} = F_{i,N}$$

$$\Rightarrow -\frac{EU_{i-1,N} - 2EU_{i,N}}{h^2} - \frac{EU_{i,N} - EU_{i,N-1}}{h^2} = RF_{i,N}$$

$$\Rightarrow EU_{i,N} = \frac{1}{3}EU_{i+1,N} + \frac{1}{3}EU_{i,N-1} + \frac{1}{3}h^2RF_{i,N}$$

$$j = 1, i = N + 1,$$

$$U_{N+1,N} = 0 \Rightarrow EU_{N+1,N} = RF_{N+1,N}$$

再计算

$$U_{half} = U + EU$$

对V, 同理地进行更新, 得 V_{half}

step2: 再更新压力

总体地, 采用内部单元→顶边→底边→左边→右边→顶点的顺序更新压力

这里, 与讲义中不同的是, 将待求解的方程写做

$$\begin{pmatrix} A & B \\ B^T & 0 \end{pmatrix} \begin{pmatrix} X \\ P \end{pmatrix} = \begin{pmatrix} Y \\ Z \neq 0 \end{pmatrix}$$

计算梯度方程残量时, 将Z计入(此处参考提示3)

(内部单元) $i : 2 : N - 1, j : 2 : N - 1$

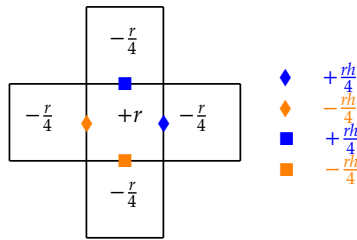


图 2: 内部单元(此处图片来自讲义)

计算散度方程残量

$$r_{i,j} = -\frac{U_{i+1,j}^{half} - U_{i,j}^{half}}{h} - \frac{V_{i,j+1}^{half} - V_{i,j}^{half}}{h} - Z_{i,j}$$

$$\delta = \frac{r_{i,j}h}{4}$$

更新内部单元速度

$$U_{i,j}^{half} = U_{i,j}^{half} - \delta, \quad U_{i+1,j}^{half} = U_{i+1,j}^{half} + \delta$$

$$V_{i,j}^{half} = V_{i,j}^{half} - \delta, \quad V_{i,j+1}^{half} = V_{i,j+1}^{half} + \delta$$

更新内部单元压力

$$P_{i,j} = P_{i,j} + r \quad P_{i+1,j} = P_{i+1,j} - \frac{1}{4}r, \quad P_{i-1,j} = P_{i-1,j} - \frac{1}{4}r$$

$$P_{i,j+1} = P_{i,j+1} - \frac{1}{4}r, \quad P_{i,j-1} = P_{i,j-1} - \frac{1}{4}r$$

(顶边) $i = 1 : N$

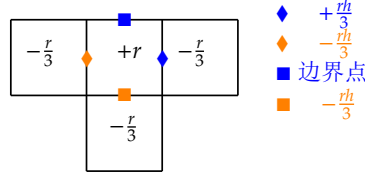


图 3: 边界单元(此处图片来自讲义)

计算散度方程残量

$$r_{i,1} = -\frac{U_{i+1,1}^{half} - U_{i,1}^{half}}{h} - \frac{V_{i,2}^{half} - V_{i,1}^{half}}{h} - Z_{i,1}$$

$$\delta = \frac{r_{i,1}h}{3}$$

更新内部单元速度

$$U_{i,1}^{half} = U_{i,1}^{half} - \delta, \quad U_{i+1,1}^{half} = U_{i+1,1}^{half} + \delta$$

$$V_{i,2}^{half} = V_{i,2}^{half} + \delta$$

更新内部单元压力

$$P_{i,1} = P_{i,1} + r \quad P_{i+1,1} = P_{i+1,1} - \frac{1}{3}r$$

$$P_{i-1,1} = P_{i-1,1} - \frac{1}{3}r, \quad P_{i,j+1} = P_{i,j+1} - \frac{1}{3}r$$

对底边,左边, 右边, 同理地进行更新

(顶点) $i = 1, j = 1$

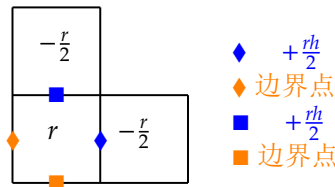


图 4: 顶点单元(此处图片来自讲义)

计算散度方程残量

$$r_{1,1} = -\frac{U_{2,1}^{half} - U_{1,1}^{half}}{h} - \frac{V_{1,2}^{half} - V_{1,1}^{half}}{h} - Z_{1,1}$$

$$\delta = \frac{r_{1,1}h}{2}$$

更新内部单元速度

$$U_{2,1}^{half} = U_{2,1}^{half} + \delta \quad V_{1,2}^{half} = V_{1,2}^{half} + \delta$$

更新内部单元压力

$$P_{1,1} = P_{1,1} + r \quad P_{2,1} = P_{2,1} - \frac{1}{2}r, \quad P_{1,2} = P_{1,2} - \frac{1}{2}r$$

对另外三个顶点, 同理地进行更新

2.1.3 限制算子和提升算子

采用如图所示的限制算子和提升算子

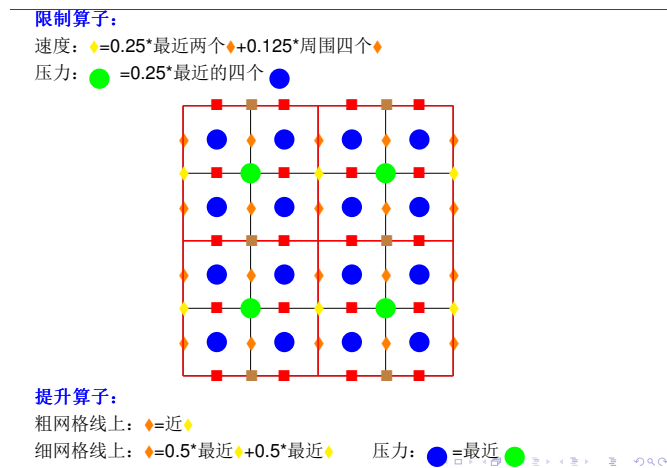


图 5: 限制算子和提升算子(此处图片来自讲义)

2.2 具体算法实现

表 1: 辅助函数

文件	作用
p1:/initialize.m	初始化 F, G 及计算真解
p1:/left_multiply_matrix.m	对矩阵 U, V 的向量形式左乘 $[A, B; B^T, 0]$ 后化回矩阵形式
p1:/limit.m	对 F, G, Z 应用限制算子, 保持其矩阵形式
p1:/lift.m	对 U, V, P 应用提升算子, 保持其矩阵形式

Algorithm 1 V-cycle 多重网格法(p1:/V_cycle.m)

```
1:  $U, V, P = 0, iter1 = \log_2(N) - \log_2(L)$ 
2: while  $r_h/r_0 > 10^{-8}$  and  $iter < MAXITER$  do
3:    $iter = iter + 1$ 
4:   for  $i = 1 : iter1$  do
5:     for  $k = 1 : v_1$  do
6:        $[U, V, P] = \text{DGS}(U, V, P, F, G, Z, N)$ 
7:     end for
8:      $store\ U, V, P, F, G, Z, N$ 
9:      $[U1, V1, P1] = \text{left\_multiply\_matrix}(U, V, P, N)$ 
10:     $[F, G, Z] = \text{limit}(F - U1, G - V1, Z - P1, N)$ 
11:     $N = N/2$ 
12:  end for
13:   $solve\ U, V, P, F, G, Z \rightarrow U, V, P$ 
14:  for  $i = iter1 : -1 : 1$  do
15:     $[U, V, P] = \text{lift}(U, V, P, N)$ 
16:     $N = N * 2$ 
17:     $U, V, P = store + U, V, P$ 
18:    for  $k = 1 : v_2$  do
19:       $[U, V, P] = \text{DGS}(U, V, P, Fstore, Gstore, Zstore, N)$ 
20:    end for
21:    if  $i == 1$  then
22:       $store\ U, V, P$ 
23:    end if
24:  end for
25: end while
```

Algorithm 2 DGS迭代法(p1:/DGS.m)

```
1: [U1, V1, ] = left_multiply_matrix(U, V, P, N)
2: RF = F - U1, RV = G - V1
3: Gauss - Seidel → EU, EV
4: U_half = U + EU, V_half = V + EV
5: for i = 2 : N - 1 do
6:   for j = 2 : N - 1 do
7:     update P(i, j)
8:   end for
9: end for
10: for i = 2 : N - 1 do
11:   update P(i, 1), P(i, N)
12: end for
13: for j = 2 : N - 1 do
14:   update P(1, j), P(N, j)
15: end for
16: update P(1, 1), P(1, N), P(N, 1), P(N, N)
```

2.3 数值结果与分析

分别取 $N = 64, 128, 256, 512, 1024, 2048$, 以 DGS 为磨光子, 用基于 V-cycle 的多重网格方法求解离散问题, 停机标准为 $\|r_h\|_2 / \|r_0\|_2 < 10^{-8}$, 对不同的 v_1, v_2, L , 比较 V-cycle 的次数和 CPU 时间, 并计算误差

$$e_N = h \left(\sum_{j=1}^N \sum_{i=1}^{N-1} \left| u_{i,j-\frac{1}{2}} - u(x_i, y_{j-\frac{1}{2}}) \right|^2 + \sum_{j=1}^{N-1} \sum_{i=1}^N \left| v_{i-\frac{1}{2},j} - v(x_{i-\frac{1}{2}}, y_j) \right|^2 \right)$$

其中 $x_i = i \frac{1}{N}, y_j = j \frac{1}{N}$, $u(i, j), v(i, j)$ 分别为计算结果在 (x_i, y_j) 处的取值
结果如下

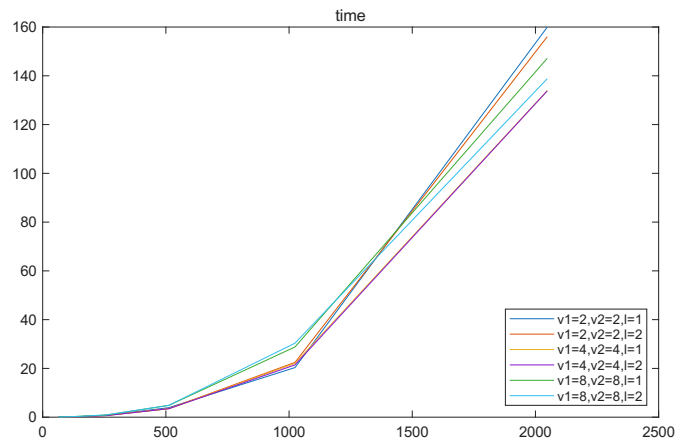


图 6: 时间随N的变化

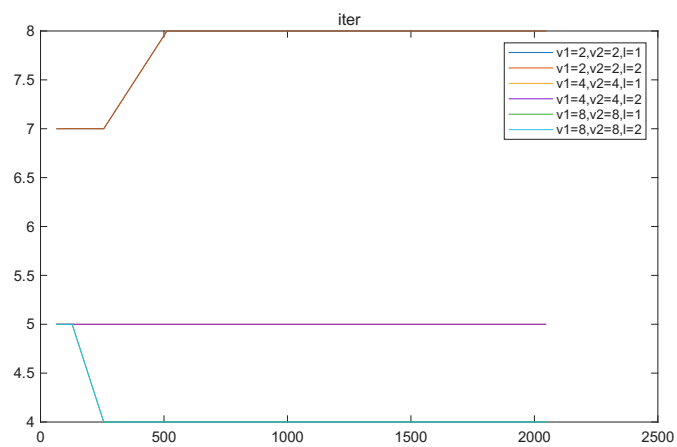


图 7: V-cycle 次数随N的变化

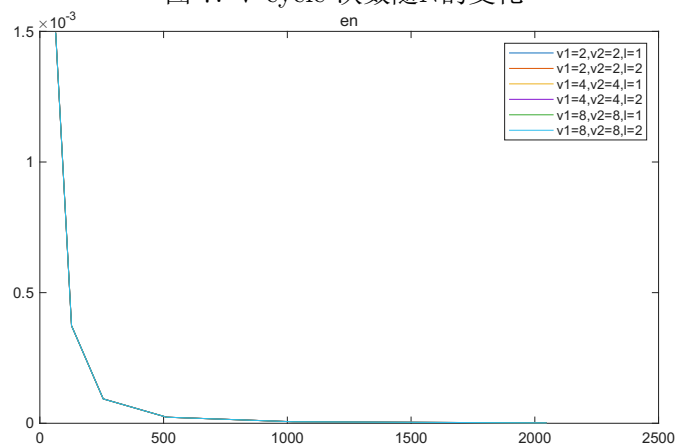


图 8: en随N的变化

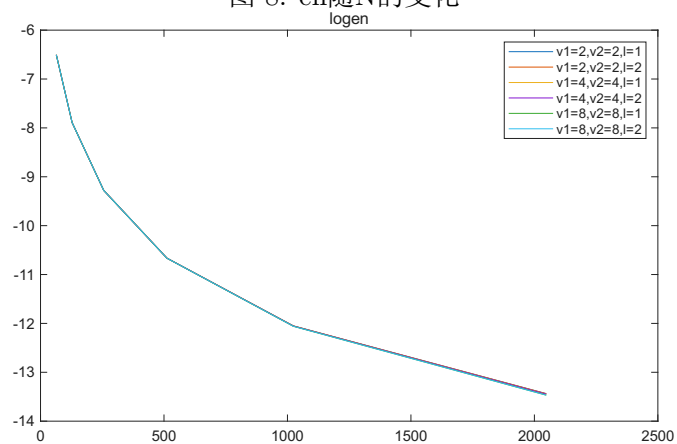


图 9: log(en)随N的变化

可见 e_N 收敛, 及在本题中不同参数计算速度差距不大, v_1, v_2 较大时, V-cycle次数较小
将部分完整数据以表格形式呈现如下

表 2: 不同参数下时间随N的变化

参数	N=64	N=128	N=256	N=512	N=1024	N=2048
$v_1 = 2, v_2 = 2, l = 1$	0.0660	0.1386	0.7367	3.9070	20.3413	159.9633
$v_1 = 2, v_2 = 2, l = 2$	0.0684	0.1458	0.6295	3.5230	22.4530	156.0362
$v_1 = 4, v_2 = 4, l = 1$	0.0494	0.1667	0.7907	3.4477	22.0449	133.9093
$v_1 = 4, v_2 = 4, l = 2$	0.0478	0.1690	0.6362	3.4793	21.4368	133.8237
$v_1 = 8, v_2 = 8, l = 1$	0.0772	0.2732	0.8475	4.8484	28.7869	147.1572
$v_1 = 8, v_2 = 8, l = 2$	0.0716	0.2549	0.9194	4.8578	30.3523	138.8127

表 3: 不同参数下 V-cycle 次数随N的变化

参数	N=64	N=128	N=256	N=512	N=1024	N=2048
$v_1 = 2, v_2 = 2, l = 1$	7	7	7	8	8	8
$v_1 = 2, v_2 = 2, l = 2$	7	7	7	8	8	8
$v_1 = 4, v_2 = 4, l = 1$	5	5	5	5	5	5
$v_1 = 4, v_2 = 4, l = 2$	5	5	5	5	5	5
$v_1 = 8, v_2 = 8, l = 1$	5	5	4	4	4	4
$v_1 = 8, v_2 = 8, l = 2$	5	5	4	4	4	4

3 P2: 使用 Uzawa Iteraion Method

3.1 数值方法

3.1.1 实用共轭梯度法

A 对称正定, 解方程

$$Ax = b$$

这等价于求

$$\arg \min_{x \in \mathbb{R}^n} \psi(x) = x^T Ax - 2b^T x$$

step1: 选定初始向量 x_0 , 下降方向为

$$p_0 = r_0 = b - Ax_0$$

并记

$$\rho_0 = r_0^T r_0, \quad w_0 = Ap_0$$

取最大下降步长

$$\alpha_0 = \frac{r_0^T r_0}{p_0^T Ap_0} = \frac{\rho_0}{p_0^T w_0}$$

得

$$x_1 = x_0 + \alpha_0 p_0, \quad r_1 = b - Ax_1 = r_0 - \alpha_0 w_0$$

step(k+1): 在平面

$$\pi = \{x | x = x_k + \xi r_k + \eta p_{k-1}, \xi, \eta \in \mathbb{R}\}$$

内找出使 ψ 下降最快的方向

$$p_k = r_k + \beta_{k-1} p_{k-1}, \quad w_k = Ap_k$$

为新的下降方向, 其中

$$\beta_{k-1} = -\frac{r_k^T Ap_{k-1}}{p_{k-1}^T Ap_{k-1}} = \frac{r_k^T r_k}{r_{k-1}^T r_{k-1}} = \frac{\rho_k}{\rho_{k-1}}$$

并取最大下降步长

$$\alpha_k = \frac{r_k^T p_k}{p_k^T Ap_k} = \frac{r_k^T r_k}{p_k^T Ap_k} = \frac{\rho_k}{p_k^T w_k}$$

$$x_{k+1} = x_k + \alpha_k p_k, \quad r_{k+1} = b - Ax_{k+1} = r_k - \alpha_k w_k, \quad \rho_{k+1} = r_{k+1}^T r_{k+1}$$

共轭梯度法一定收敛到方程的真解, 取停止条件为

$$\sqrt{\rho} < \epsilon \|b\|_2 \quad \text{or} \quad k \geq k_{max}$$

3.1.2 Uzawa Iteraion Method

求解线性方程

$$\begin{pmatrix} A & B \\ B^T & 0 \end{pmatrix} \begin{pmatrix} X \\ P \end{pmatrix} = \begin{pmatrix} Y \\ 0 \end{pmatrix}$$

step1: 初值取为 $U, V, P = 0$

step2: 精确求解

$$AX_{k+1} = Y - B^T P_k$$

这里采用实用共轭梯度法

step3: 更新压力

$$P_{k+1} = P_k + \alpha(B^T X_k)$$

step4:

$$r_h = \begin{pmatrix} Y \\ 0 \end{pmatrix} - \begin{pmatrix} A & B \\ B^T & 0 \end{pmatrix} \begin{pmatrix} \hat{X} \\ \hat{P} \end{pmatrix}$$

$$r_0 = Y$$

停止条件为

$$\frac{\|r_h\|_2}{\|r_0\|_2} < 10^{-8}$$

如满足停止条件, 停止迭代, $\hat{X}$ 为近似解

否则, 回到step2

3.2 具体算法实现

表 4: 辅助函数

文件	作用
p2:/initialize.m	初始化 F, G 及计算真解
p2:/left_multiply_matrix.m	对矩阵 U, V 的向量形式左乘 $[A, B; B^T, 0]$ 后化回矩阵形式
p2:/left_multiply_A.m	对矩阵 U, V 的向量形式左乘 A 后化回矩阵形式
p2:/left_multiply_B.m	对矩阵 P 的向量形式左乘 B 后化回矩阵形式
p2:/left_multiply_BT.m	对矩阵 U, V 的向量形式左乘 B^T 后化回矩阵形式
p2:/dot_multiply.m	对两个矩阵, 化成向量形式后求内积

Algorithm 3 Uzawa Iteration Method(p2:/Uzawa.m)

- 1: $P = 0, r_0 = \sqrt{\|F\|_F^2 + \|G\|_F^2}$
 - 2: **while** $r_h/r_0 > 10^{-8}$ **and** $iter < MAXITER$ **do**
 - 3: $[F1, G1] = \text{left_multiply_B}(P, N)$
 - 4: $[U, V, iter1] = \text{CG}(F - F1, G - G1, N, 10^{-8})$
 - 5: $P = P + \alpha \text{left_multiply_BT}(U, V, N)$
 - 6: $[U1, V1, P1] = \text{left_multiply_matrix}(U, V, P, N)$
 - 7: $r_h = \sqrt{\|U1 - F\|_F^2 + \|V1 - G\|_F^2 + \|P1\|_F^2}$
 - 8: **end while**
-

Algorithm 4 实用共轭梯度法(p2:/CG.m)

```
1:  $U, V, P = 0, RU = F, RV = G, b = \|[F, G]\|_F, MAXITER = N\sqrt{N}$ 
2: while  $\sqrt{\rho} > \epsilon$  and  $iter < MAXITER$  do
3:    $iter = iter + 1$ 
4:   if  $i == 1$  then
5:      $PU = RU, PV = RV$ 
6:      $\rho = \|RU\|_F^2 + \|RV\|_F^2$ 
7:   else
8:      $\hat{\rho} = \rho$ 
9:      $\rho = \|RU\|_F^2 + \|RV\|_F^2$ 
10:     $\beta = \rho / \hat{\rho}$ 
11:     $PU = RU + \beta PU; PV = RV + \beta PV$ 
12:  end if
13:   $[WU, WV] = \text{left\_multiply\_A}(PU, PV, N);$ 
14:   $\alpha = \rho / (\text{dot\_multiply}(WU, PU) + \text{dot\_multiply}(WV, PV))$ 
15:   $U = U + \alpha PU, V = V + \alpha PV$ 
16:   $RU = RU - \alpha WU, RV = RV - \alpha WV$ 
17: end while
```

3.3 数值结果

分别取 $N = 64, 128, 256, 512$, 以 Uzawa Iteration Method 求解离散问题, 停机标准为 $\|r_h\|_2 / \|r_0\|_2 < 10^{-8}$, 并计算误差

$$e_N = h \left(\sum_{j=1}^N \sum_{i=1}^{N-1} \left| u_{i,j-\frac{1}{2}} - u(x_i, y_{j-\frac{1}{2}}) \right|^2 + \sum_{j=1}^{N-1} \sum_{i=1}^N \left| v_{i-\frac{1}{2},j} - v(x_{i-\frac{1}{2}}, y_j) \right|^2 \right)$$

其中 $x_i = i \frac{1}{N}, y_j = j \frac{1}{N}$, $u(i, j), v(i, j)$ 分别为计算结果在 (x_i, y_j) 处的取值

取 $\alpha = 1$, 结果如下

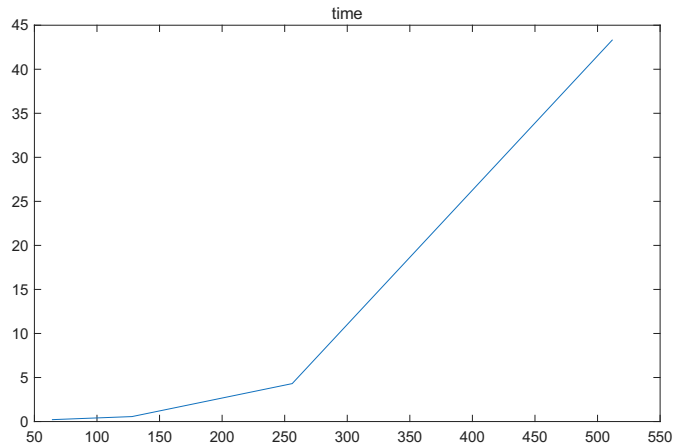


图 10: 时间随N的变化

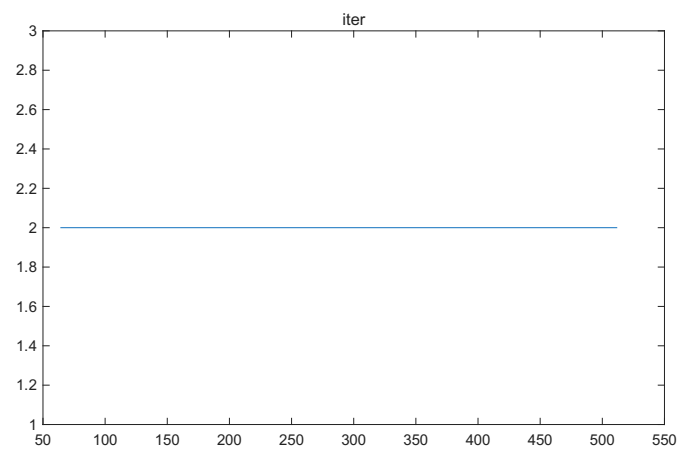


图 11: 外循环次数随N的变化

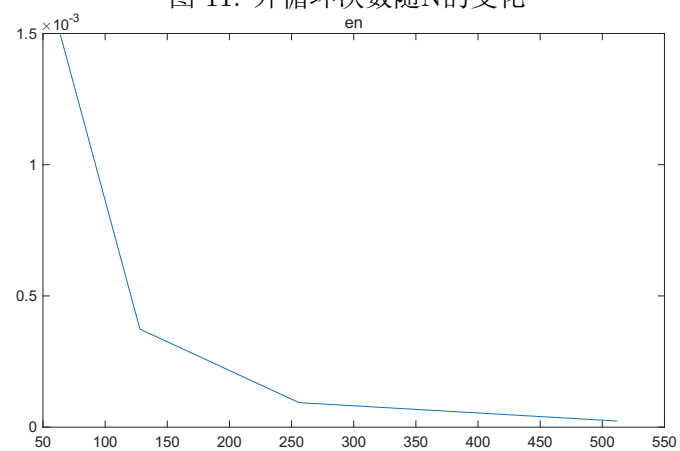


图 12: en随N的变化

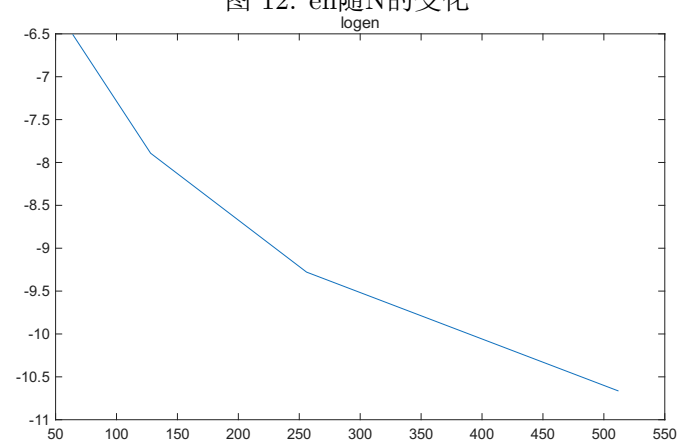


图 13: log(en)随N的变化

4 P3: 使用 Inexact Uzawa Iteration Method

4.1 数值方法

4.1.1 预优共轭梯度法

A 对称正定, 解方程

$$Ax = b,$$

共轭梯度法的收敛速度为

$$\|x_k - x_*\|_A \leq 2 \left(\frac{\sqrt{\kappa_2} - 1}{\sqrt{\kappa_2} + 1} \right)^k \|x_0 - x_*\|$$

解原方程等价于解

$$\tilde{A}\tilde{x} = \tilde{b}$$

其中

$$\tilde{A} = C^{-1}AC^{-1}, \quad \tilde{x} = Cx, \quad \tilde{b} = C^{-1}b$$

记

$$M = C^2$$

当 $\|\tilde{A}\|_2$ 较小时, 对前述方程使用共轭梯度法求解, 收敛速度较快, 可知 M 为 A 的近似时收敛较快

在共轭梯度法的基础上, 给出预优共轭梯度法, 设对于新方程采用共轭梯度法求解时, 得到的各个中间变量为

$$\tilde{x}_k, \tilde{p}_k, \tilde{r}_k, \tilde{\alpha}_k, \tilde{\beta}_k, \tilde{\rho}_k, \tilde{w}_k$$

step1: 选定初始向量

$$x_0 = C^{-1}\tilde{x}_0$$

$$r_0 = b - Ax_0 = C\tilde{b} - C\tilde{A}C \cdot C^{-1}\tilde{x}_0 = C\tilde{r}_0$$

$$p_0 = z_0 = M^{-1}r_0 = C\tilde{p}_0$$

$$\rho_0 = r_0^T z_0 = \tilde{r}_0^T \tilde{r}_0 = \tilde{\rho}_0$$

step(k+2): 在求解过程中

$$w_k = Ap_k = C\tilde{A}\tilde{p}_k$$

$$\alpha_k = \frac{\rho_k}{p_k^T w_k} = \frac{\tilde{r}_k^T \tilde{r}_k}{\tilde{p}_k^T \tilde{A}\tilde{p}_k} = \tilde{\alpha}_k$$

$$x_{k+1} = x_k + \alpha_k p_k = C^{-1}x_{k+1}$$

$$r_{k+1} = r_k - \alpha_k w_k = C\tilde{r}_{k+1}$$

$$z_{k+1} = M^{-1}r_{k+1} = C^{-1}r_{k+1} = C\tilde{r}_{k+1}$$

$$\rho_{k+1} = r_{k+1}^T z_{k+1} = \tilde{r}_{k+1}^T \tilde{r}_{k+1} = \tilde{\rho}_{k+1}$$

$$\beta_k = \frac{\rho_{k+1}}{\rho_k} = \tilde{\beta}_k$$

$$p_{k+1} = z_{k+1} + \beta_k p_k = C^{-1}p_{k+1}$$

对新方程使用共轭梯度法, 收敛到其真解 $\tilde{x}_*$

所以上述预优共轭梯度法收敛到原方程的真解 x_*

取停止条件为

$$\sqrt{\rho} < \epsilon \|b\|_2 \quad or \quad k \geq k_{max}$$

4.1.2 Inexact Uzawa Iteraion Method

求解线性方程

$$\begin{pmatrix} A & B \\ B^T & 0 \end{pmatrix} \begin{pmatrix} X \\ P \end{pmatrix} = \begin{pmatrix} Y \\ 0 \end{pmatrix}$$

step1: 初值取为 $U, V, P = 0$

step2: 采取预优共轭梯度法, 近似求解

$$AX_{k+1} = Y - B^T P_k$$

在本问题中, 使用 V-cycle 多重网格方法为预条件子

即在采用预优共轭梯度法时, 在计算

$$Mz = r$$

的过程中, 采用 V-cycle 法近似求解

$$Az = r$$

step3: 更新压力

$$P_{k+1} = P_k + \alpha(B^T X_k)$$

step4:

$$r_h = \begin{pmatrix} Y \\ 0 \end{pmatrix} - \begin{pmatrix} A & B \\ B^T & 0 \end{pmatrix} \begin{pmatrix} \hat{X} \\ \hat{P} \end{pmatrix}$$

$$r_0 = Y$$

停止条件为

$$\frac{\|r_h\|_2}{\|r_0\|_2} < 10^{-8}$$

如满足停止条件, 停止迭代, $\hat{X}$ 为近似解

否则, 回到step2

4.2 具体算法实现

表 5: 辅助函数

文件	作用
p3:/initialize.m	初始化 F, G 及计算真解
p3:/left_multiply_matrix.m	对矩阵 U, V 的向量形式左乘 $[A, B; B^T, 0]$ 后化回矩阵形式
p3:/left_multiply_A.m	对矩阵 U, V 的向量形式左乘 A 后化回矩阵形式
p3:/left_multiply_B.m	对矩阵 P 的向量形式左乘 B 后化回矩阵形式
p3:/left_multiply_BT.m	对矩阵 U, V 的向量形式左乘 B^T 后化回矩阵形式
p3:/dot_multiply.m	对两个矩阵, 化成向量形式后求内积
p3:/limit.m	对 F, G 应用限制算子, 保持其矩阵形式
p3:/lift.m	对 U, V 应用提升算子, 保持其矩阵形式
p3:/GS.m	对 $AX = Y$ 做Gauss-Seidel迭代

Algorithm 5 V-cycle 多重网格法(p3:/V_cycle.m)

```
1:  $U, V = 0, iter1 = \log_2(N) - \log_2(L)$ 
2: while  $r_h/r_0 > 10^{-8}$  and  $iter < MAXITER$  do
3:    $iter = iter + 1$ 
4:   for  $i = 1 : iter1$  do
5:     for  $k = 1 : v_1$  do
6:        $[U, V] = \mathbf{GS}(U, V, F, G, N)$ 
7:     end for
8:      $store\ U, V, F, G, N$ 
9:      $[U1, V1] = \mathbf{left\_multiply\_A}(U, V, N)$ 
10:     $[F, G] = \mathbf{limit}(F - U1, G - V1, N)$ 
11:     $N = N/2$ 
12:  end for
13:   $solve\ U, V, F, G \rightarrow U, V$ 
14:  for  $i = iter1 : -1 : 1$  do
15:     $[U, V] = \mathbf{lift}(U, V, N)$ 
16:     $N = N * 2$ 
17:     $U, V = store + U, V$ 
18:    for  $k = 1 : v_2$  do
19:       $[U, V] = \mathbf{GS}(U, V, Fstore, Gstore, N)$ 
20:    end for
21:    if  $i == 1$  then
22:       $store\ U, V$ 
23:    end if
24:  end for
25: end while
```

Algorithm 6 预优共轭梯度法(p3:/PCG.m)

```
1:  $U, V, P = 0, RU = F, RV = G, b = \|[F, G]\|_F, MAXITER = N$ 
2: while  $\sqrt{\rho} > \epsilon b (\epsilon = 10^{-8})$  and  $iter < MAXITER$  do
3:    $iter = iter + 1$ 
4:    $[ZU, ZV] = \mathbf{V\_cycle}(RU, RV, v_1, v_2, N)$ 
5:   if  $i == 1$  then
6:      $PU = ZU, PV = ZV$ 
7:      $\rho = \mathbf{dot\_multiply}(RU, ZU) + \mathbf{dot\_multiply}(RV, ZV)$ 
8:   else
9:      $\hat{\rho} = \rho$ 
10:     $\rho = \mathbf{dot\_multiply}(RU, ZU) + \mathbf{dot\_multiply}(RV, ZV)$ 
11:     $\beta = \rho / \hat{\rho}$ 
12:     $PU = ZU + \beta PU, PV = ZV + \beta PV$ 
13:  end if
14:   $[WU, WV] = \mathbf{left\_multiply\_A}(PU, PV, N);$ 
15:   $\alpha = \rho / (\mathbf{dot\_multiply}(WU, PU) + \mathbf{dot\_multiply}(WV, PV))$ 
16:   $U = U + \alpha PU, V = V + \alpha PV$ 
17:   $RU = RU - \alpha WU, RV = RV - \alpha WV$ 
18: end while
```

4.3 数值结果及相应分析

分别取 $N = 64, 128, 256, 512, 1024, 2048$, 以 Inexact Uzawa Iteration Method 求解离散问题, 停机标准为 $\|r_h\|_2 / \|r_0\|_2 < 10^{-8}$, 其中以 V-cycle 多重网格方法为预条件子, 利用共轭梯度法求解每一步的子问题 $\mathbf{A}\mathbf{U}_{k+1} = \mathbf{F} - \mathbf{B}\mathbf{P}_k$, 对不同的 $\alpha, \tau, 1, 2, L$, 比较外循环的迭代次数和 CPU 时间, 并计算误差

$$e_N = h \left(\sum_{j=1}^N \sum_{i=1}^{N-1} \left| u_{i,j-\frac{1}{2}} - u(x_i, y_{j-\frac{1}{2}}) \right|^2 + \sum_{j=1}^{N-1} \sum_{i=1}^N \left| v_{i-\frac{1}{2},j} - v(x_{i-\frac{1}{2}}, y_j) \right|^2 \right)$$

其中 $x_i = i \frac{1}{N}, y_j = j \frac{1}{N}$, $u(i, j), v(i, j)$ 分别为计算结果在 (x_i, y_j) 处的取值结果如下

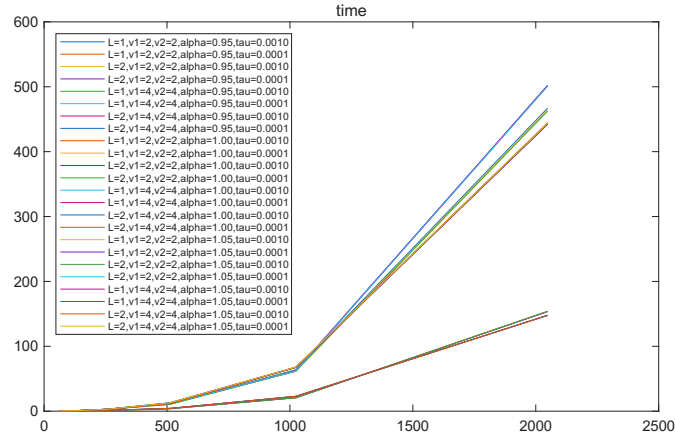


图 14: 不同参数下时间随N的变化

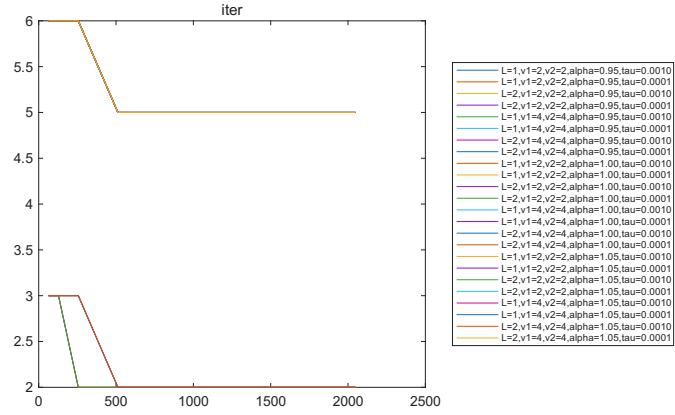


图 15: 不同参数下外循环次数随N的变化

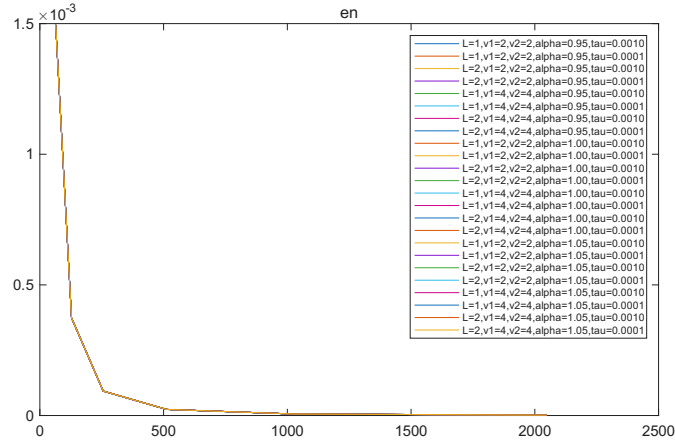


图 16: 不同参数下en随N的变化

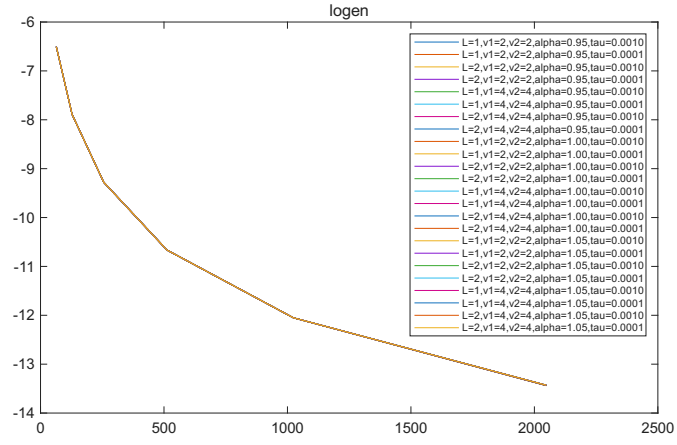


图 17: 不同参数下 $\log(en)$ 随 N 的变化

可见 e_N 收敛, $\alpha = 1$ 时收敛速度显著快于 $\alpha = 0.95$ or 1.05 , 且迭代次数更小, 而其余参数对结果影响不大
将部分数据用表格形式完整呈现如下

表 6: 不同参数下时间随N的变化

参数	N=64	N=128	N=256	N=512	N=1024	N=2048
$L = 1, v_1 = 2, v_2 = 2, \alpha = 0.95, \tau = 0.0010$	0.6831	0.9120	2.9944	10.9916	63.9492	466.9588
$L = 1, v_1 = 2, v_2 = 2, \alpha = 0.95, \tau = 0.0001$	0.2831	0.7416	3.0426	11.5018	62.4431	501.3319
$L = 2, v_1 = 2, v_2 = 2, \alpha = 0.95, \tau = 0.0010$	0.2783	0.7028	2.7757	10.7246	61.6444	464.2096
$L = 2, v_1 = 2, v_2 = 2, \alpha = 0.95, \tau = 0.0001$	0.2646	0.7813	3.0126	11.3898	62.1861	502.1172
$L = 1, v_1 = 4, v_2 = 4, \alpha = 0.95, \tau = 0.0010$	0.2721	0.6915	2.7541	11.9746	67.7576	443.8745
$L = 1, v_1 = 4, v_2 = 4, \alpha = 0.95, \tau = 0.0001$	0.2989	0.7732	2.9550	13.0916	67.5270	442.6019
$L = 2, v_1 = 4, v_2 = 4, \alpha = 0.95, \tau = 0.0010$	0.2535	0.7392	2.7973	11.7912	67.7846	442.1117
$L = 2, v_1 = 4, v_2 = 4, \alpha = 0.95, \tau = 0.0001$	0.2854	0.8095	3.0190	12.7961	67.7756	442.6326
$L = 1, v_1 = 2, v_2 = 2, \alpha = 1.00, \tau = 0.0010$	0.1061	0.2705	0.7864	3.8139	20.8214	153.8682
$L = 1, v_1 = 2, v_2 = 2, \alpha = 1.00, \tau = 0.0001$	0.1284	0.3217	0.8116	3.8405	20.7670	153.7367
$L = 2, v_1 = 2, v_2 = 2, \alpha = 1.00, \tau = 0.0010$	0.1019	0.2776	0.8051	3.9084	20.6553	153.4189
$L = 2, v_1 = 2, v_2 = 2, \alpha = 1.00, \tau = 0.0001$	0.1265	0.3093	0.8189	3.8054	20.6308	153.8722
$L = 1, v_1 = 4, v_2 = 4, \alpha = 1.00, \tau = 0.0010$	0.1035	0.2684	1.0926	4.3201	22.7530	147.3751
$L = 1, v_1 = 4, v_2 = 4, \alpha = 1.00, \tau = 0.0001$	0.1187	0.3329	1.2836	4.2956	23.0500	147.6945
$L = 2, v_1 = 4, v_2 = 4, \alpha = 1.00, \tau = 0.0010$	0.1123	0.2875	1.1047	4.2912	22.8108	147.9644
$L = 2, v_1 = 4, v_2 = 4, \alpha = 1.00, \tau = 0.0001$	0.1234	0.3295	1.2970	4.3109	22.8637	147.5623
$L = 1, v_1 = 2, v_2 = 2, \alpha = 1.05, \tau = 0.0010$	0.3170	0.7805	3.0682	10.6040	62.0672	463.2114
$L = 1, v_1 = 2, v_2 = 2, \alpha = 1.05, \tau = 0.0001$	0.3073	0.7054	2.9982	11.4006	61.7156	501.4783
$L = 2, v_1 = 2, v_2 = 2, \alpha = 1.05, \tau = 0.0010$	0.2919	0.7097	2.7750	10.5026	61.7900	462.4892
$L = 2, v_1 = 2, v_2 = 2, \alpha = 1.05, \tau = 0.0001$	0.2706	0.7195	2.9506	11.4420	62.0211	500.9380
$L = 1, v_1 = 4, v_2 = 4, \alpha = 1.05, \tau = 0.0010$	0.2567	0.7130	2.8251	11.8441	67.4739	442.9305
$L = 1, v_1 = 4, v_2 = 4, \alpha = 1.05, \tau = 0.0001$	0.3051	0.7647	2.9605	12.8265	67.5050	443.1289
$L = 2, v_1 = 4, v_2 = 4, \alpha = 1.05, \tau = 0.0010$	0.2489	0.7205	2.8042	11.7245	67.5906	444.3659
$L = 2, v_1 = 4, v_2 = 4, \alpha = 1.05, \tau = 0.0001$	0.2905	0.7561	2.9851	12.8375	68.0027	443.6839

表 7: 不同参数下外循环次数随N的变化

参数	N=64	N=128	N=256	N=512	N=1024	N=2048
$L = 1, v_1 = 2, v_2 = 2, \alpha = 0.95, \tau = 0.0010$	6	6	6	5	5	5
$L = 1, v_1 = 2, v_2 = 2, \alpha = 0.95, \tau = 0.0001$	6	6	6	5	5	5
$L = 2, v_1 = 2, v_2 = 2, \alpha = 0.95, \tau = 0.0010$	6	6	6	5	5	5
$L = 2, v_1 = 2, v_2 = 2, \alpha = 0.95, \tau = 0.0001$	6	6	6	5	5	5
$L = 1, v_1 = 4, v_2 = 4, \alpha = 0.95, \tau = 0.0010$	6	6	6	5	5	5
$L = 1, v_1 = 4, v_2 = 4, \alpha = 0.95, \tau = 0.0001$	6	6	6	5	5	5
$L = 2, v_1 = 4, v_2 = 4, \alpha = 0.95, \tau = 0.0010$	6	6	6	5	5	5
$L = 2, v_1 = 4, v_2 = 4, \alpha = 0.95, \tau = 0.0001$	6	6	6	5	5	5
$L = 1, v_1 = 2, v_2 = 2, \alpha = 1.00, \tau = 0.0010$	3	3	2	2	2	2
$L = 1, v_1 = 2, v_2 = 2, \alpha = 1.00, \tau = 0.0001$	3	3	2	2	2	2
$L = 2, v_1 = 2, v_2 = 2, \alpha = 1.00, \tau = 0.0010$	3	3	2	2	2	2
$L = 2, v_1 = 2, v_2 = 2, \alpha = 1.00, \tau = 0.0001$	3	3	2	2	2	2
$L = 1, v_1 = 4, v_2 = 4, \alpha = 1.00, \tau = 0.0010$	3	3	3	2	2	2
$L = 1, v_1 = 4, v_2 = 4, \alpha = 1.00, \tau = 0.0001$	3	3	3	2	2	2
$L = 2, v_1 = 4, v_2 = 4, \alpha = 1.00, \tau = 0.0010$	3	3	3	2	2	2
$L = 2, v_1 = 4, v_2 = 4, \alpha = 1.00, \tau = 0.0001$	3	3	3	2	2	2
$L = 1, v_1 = 2, v_2 = 2, \alpha = 1.05, \tau = 0.0010$	6	6	6	5	5	5
$L = 1, v_1 = 2, v_2 = 2, \alpha = 1.05, \tau = 0.0001$	6	6	6	5	5	5
$L = 2, v_1 = 2, v_2 = 2, \alpha = 1.05, \tau = 0.0010$	6	6	6	5	5	5
$L = 2, v_1 = 2, v_2 = 2, \alpha = 1.05, \tau = 0.0001$	6	6	6	5	5	5
$L = 1, v_1 = 4, v_2 = 4, \alpha = 1.05, \tau = 0.0010$	6	6	6	5	5	5
$L = 1, v_1 = 4, v_2 = 4, \alpha = 1.05, \tau = 0.0001$	6	6	6	5	5	5
$L = 2, v_1 = 4, v_2 = 4, \alpha = 1.05, \tau = 0.0010$	6	6	6	5	5	5
$L = 2, v_1 = 4, v_2 = 4, \alpha = 1.05, \tau = 0.0001$	6	6	6	5	5	5